

MODERN C++

# Top 5 Features

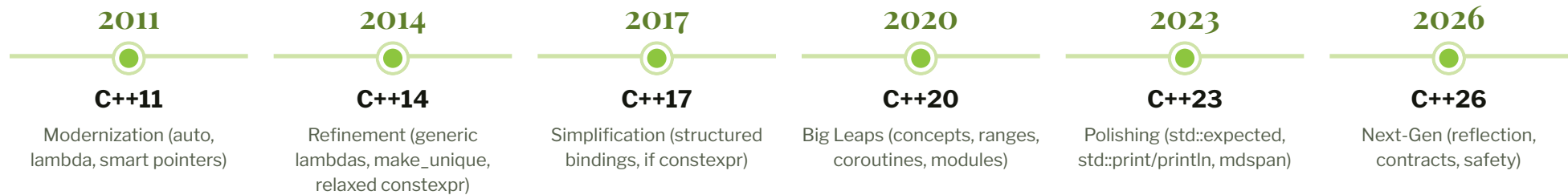
for each version

---

SOURCE · [github.com/CodingInLondon/cpp-katas](https://github.com/CodingInLondon/cpp-katas)

VERSION · v2

## C++ Standards Evolution



If you're into C#, Python or Rust, you might wonder what it is about C++ that is "modern". Modern C++ refers to all revisions of the standard since C++11. There has been one every 3 years and the next one is C++26 (not covered here).

They brought significant changes, changes that are not trivial to learn if you grew up with C++98. Among the mind-bending concepts you will find:

- how objects are passed by default (copy vs move semantics, r-value references)
- a new way to reason about concurrency with the memory model
- atomics, memory ordering and lock-free programming.
- template programming, inlining and all the language bits that push work to compile time.

At the same time, C++ has shifted from being a general-purpose language (In the 1990's you would pick C++ MFC to build Enterprise UX) towards specialising in low-latency use cases: high-end gaming, avionics, high-frequency trading.

As a general-purpose language, C++ is a dinosaur. As a highly specialised tool, it's cutting edge.

New features make C++ faster, more explicit, and (sometimes) safer. Modern C++ is still as close to the metal as you can get and yes, **you can still shoot yourself in the foot even if you're careful.**

This document is a quick overview of the new features, **five** of which have been arbitrarily picked for each revision of the standard.

# C++11 (2011)

- 1 Move semantics** (`std::move`) – **Purpose:** Eliminate unnecessary copies for performance, enabling efficient transfer of resources.

```
std::string a = "hello";
std::string b =
std::move(a); // b takes ownership; a is valid but its value
is unspecified
```

**Use-case:** Critical in high-performance code (STL containers, large objects) to avoid deep copies; requires care when using moved-from objects.

- 3 auto type deduction** – **Purpose:** Remove repetitive type annotations. Developers love that `auto` "tracks" the right type if the initializer changes, reducing visual clutter.

```
auto x = 42; // x is int
auto vec = std::vector<double>{1.1, 2.2};
```

**Use-case:** Ideal when the exact type is obvious or long (like iterators or lambdas), making code easier to write and maintain (trade-off: hides type, so readability relies on initializer clarity).

**Trap:** Using `auto` instead of `auto&` when binding to a large object reference triggers a silent copy.

- 2 std::unique\_ptr** – **Purpose:** Modern smart pointers for automatic memory management. `unique_ptr` (introduced in C++11) makes ownership explicit and RAII-based, preventing leaks.

```
#include <memory>

struct MyClass {
    explicit MyClass(int) {}
};

std::unique_ptr<MyClass> p(new MyClass(10));
```

**Use-case:** Use instead of raw `new`; essential in modern C++ projects (e.g. resource management, container elements).

- 4 Range-based for loops** – **Purpose:** Simple iteration syntax for containers/arrays. More readable than index/pointer loops and less error-prone.

```
std::vector<int> v = {1,2,3};
for (int x : v) {
    std::cout << x << "\n"; // prints 1 2 3
}
```

**Use-case:** Preferred for iterating over any STL container or array; avoid manual iterator code. (Trade-off: cannot modify container size during iteration.)

- 5 Lambda functions** – **Purpose:** Write inline, anonymous function objects concisely. Lambdas improve code clarity by placing the function logic at the call site.

```
auto add = [](int x, int y) { return x + y; };  
std::cout << add(3, 4); // outputs 7
```

**Use-case:** Great for callbacks, custom sorting, or any small function. By capturing context, they reduce boilerplate of separate functor classes (trade-off: may complicate debugging if overused).

## C++14 (2014)

- 1** `std::make_unique` – **Purpose:** Factory function for `unique_ptr` (missing in C++11). It simplifies pointer creation and is exception-safe.

```
std::unique_ptr<Foo> ptr = std::make_unique<Foo>(arg);
```

**Use-case:** Always prefer `make_unique` over `new`; it's safer (no manual `delete` and handles exceptions).

- 3** **Return-type deduction ( `auto` function return )** – **Purpose:** Functions can omit the return type (use `auto`) and let the compiler deduce it from the return expression.

```
auto sum(int a, int b) { return a + b; } // deduces return type int
```

**Use-case:** Shortens template code and member functions in classes without specifying return, at the expense of less explicit function signatures.

- 2** **Relaxed `constexpr`** – **Purpose:** More code allowed in `constexpr` functions (loops, local vars, branching), enabling complex compile-time computation.

```
constexpr int factorial(int n) {
    int r = 1;
    for (int i = 1; i ≤ n; ++i) r *= i;
    return r;
}
constexpr int f6 = factorial(6); // computed at compile-time
```

**Use-case:** Useful in performance-critical code to compute values at compile time (e.g. lookup tables).

- 4** **Generic lambdas ( `auto` parameters )** – **Purpose:** Let a lambda deduce parameter types automatically, making lambdas templated. One lambda can handle multiple types.

```
auto f = [](auto a, auto b) { return a + b; };
std::cout << f(2, 3) << f(2.5, 4.5); // outputs 5 and 7
```

**Use-case:** Useful for writing reusable callbacks (e.g. `std::sort`) without hand-writing templates. Eliminates boilerplate lambda templates.

5

**decltype(auto) return** – **Purpose:** Deduce return type exactly as the returned expression (including references). It preserves const/ref qualifiers automatically.

```
std::string& get1() { static std::string s = "hello"; return  
s; }  
std::string get2() { return "world"; }  
decltype(auto) wrapper1() { return get1(); } // returns  
std::string&  
decltype(auto) wrapper2() { return get2(); } // returns  
std::string
```

**Use-case:** Handy for perfectly forwarding return of auto members in generic code; to maintain reference qualifiers correctly (trade-off: can be subtle if misused).

# C++17 (2017)

- 1** `std::string_view` – **Purpose:** Non-owning, read-only view of a string. It allows zero-copy passing of substrings/literals with minimal overhead.

```
void print_sv(std::string_view sv) {
    std::cout << sv;
}
print_sv("hello"); // no allocation, efficient
```

**Use-case:** Used in performance-sensitive code (parsing, logging) to avoid copying strings. Must ensure original string outlives the view.

- 3** `std::optional` – **Purpose:** Represents "maybe" a value (either containing a value or `nullopt`). It replaces ad-hoc null flags or sentinel values with a clear type.

```
std::optional<int> divide(int a, int b) {
    if (b == 0) return std::nullopt;
    return a/b;
}
```

**Use-case:** Useful in functions that may fail.

- 2** `if constexpr` – **Purpose:** Compile-time conditional. Compiler only instantiates the branch whose condition is true, removing the need for SFINAE tricks.

```
template<typename T>
void foo(T t) {
    if constexpr(std::is_integral_v<T>)
        std::cout << "Integral\n";
    else
        std::cout << "Non-integral\n";
}
```

**Use-case:** Simplifies template metaprogramming (e.g. enabling/disabling code paths). Over `#if` or `enable_if`, it produces clearer code.

- 4** `std::variant` – **Purpose:** Safe union type (holds one of several types). Ideal for functions or APIs with different return types or for a tagged union pattern (like `Boost.Variant`).

```
std::variant<int, std::string> v;
v = 10;
v = "ten"; // now holds string
```

**Use-case:** Useful in interpreters, serializers, or any code requiring polymorphic value containers.

- 5 **Structured bindings** ( `auto [a,b] = ...` ) - **Purpose:** Unpack tuples, pairs, or structs into separate variables by name. Improves readability by eliminating manual `get<>()` or `.first/.second`.

```
std::pair<int, std::string> p{42, "hello"};
auto [num, text] = p; // num=42, text="hello"
```

**Use-case:** Commonly used to unpack `map` entries or function-returned tuples. (Trade-off: structured bindings make copies unless using `auto&`.)

# C++20 (2020)

- 1 Concepts – Purpose:** Clearly specify template requirements with named constraints (e.g. `std::integral`). They provide human-readable error messages and early checks, greatly improving template usability.

```
template<std::integral T>
T add(T a, T b) { return a+b; }
```

**Use-case:** Widely used to constrain generic code (e.g. numeric algorithms). Makes templates self-documenting. (Trade-off: compiler support maturation required, but now generally available.)

- 3 Ranges Library – Purpose:** Provides views, range adaptors, and range-based algorithms. Enables pipelined STL algorithms and filters (functional style) without extra loops.

```
#include <ranges>
auto v = std::vector{1,2,3,4,5};
for (int x : v | std::views::filter([](int i){ return
i%2==0; }))
    std::cout << x; // prints 2 4
```

**Use-case:** Makes complex data processing concise (filter/map pipelines). (Trade-off: a learning curve, and potential hidden cost if laziness is misused.)

- 2 std::format – Purpose:** Safe, extensible text formatting (like Python's `format` or `fmt` library). It simplifies formatted output with `{}` placeholders.

```
std::cout << std::format("Hello {}", "world");
```

**Use-case:** Printing and logging become clearer. Many developers use `{fmt}` library similarly; having it in the standard removes an external dependency.

- 4 **Coroutines** ( `co_await` ) – **Purpose:** Language-level support for lazy/asynchronous functions. Coroutines let you write async code (or iterators) as if it were linear, hiding the state machine.

```
#include <coroutine>
#include <iostream>

struct Task {
    struct promise_type {
        Task get_return_object() {
            return Task{std::coroutine_handle<promise_type>::from_promise(*this)};
        }
        std::suspend_never initial_suspend() noexcept { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() { std::terminate(); }
    };

    std::coroutine_handle<promise_type> handle;
};

Task async_work() {
    std::cout << "Start work\n";
    co_await std::suspend_always{}; // suspend coroutine here
    std::cout << "Resume work\n";
}

int main() {
    Task t = async_work(); // runs up to the co_await, prints "Start work"
    std::cout << "Suspended, back in main\n";
    t.handle.resume(); // prints "Resume work", coroutine runs to completion
}
```

**Use-case:** Useful for async I/O, event loops, and generators. (Trade-off: complexity of promises/schedulers; debugging stacks can be non-obvious.)

- 5 **Modules – Purpose:** A new modular import system to replace textual headers. Promises faster builds (no redundant parsing) and better encapsulation.

```
// foo.cppm
export module foo;

export int add(int a, int b) {
    return a + b;
}

// main.cpp
#include <iostream>
import foo;

int main() {
    std::cout << add(2, 3);
}
```

**Use-case:** Can reduce build times in large codebases but requires toolchain support. Still seen as experimental.

# C++23 (2023)

- 1** `std::expected` – **Purpose:** Standardized the "expected or error" monad. `std::expected<T,E>` holds either a `T` or an error `E`, enabling error propagation without exceptions.

```
std::expected<int, std::string> read_number(bool ok) {
    if (!ok) return std::unexpected("Read failed");
    return 42;
}
```

**Use-case:** Common in parsing or embedded code where throwing is undesirable. Enables explicit error handling via `.has_value()` / `.error()`.

- 3** `std::mdspan` (multi-dimensional array view) – **Purpose:** Provides a view (non-owning) of multi-dimensional arrays/matrices, with native support for `operator[]` taking multiple indices.

```
#include <array>
#include <cstdint>
#include <mdspan>

std::array<double, 9> arr{};
std::mdspan<double, std::extents<std::size_t, 3, 3>>
m(arr.data());

double x = m[1, 2]; // 2D indexing
```

**Use-case:** Crucial for numeric and scientific computing where manual multi-dimensional data arithmetic is error-prone.

- 2** `std::print` / `std::println` – **Purpose:** C++23 addition for built-in printing using `std::format`-style `{}` replacement fields. Simplifies console output with type safety.

```
std::print("Hello, {}!\n", "world");
```

**Use-case:** Makes I/O concise (replacing cumbersome `<iostream>` syntax). Many devs welcome it as it parallels `fmt::print`.

- 4** `if constexpr` – **Purpose:** Enables compile-time-only branches. Useful when a function needs different code paths at compile-time vs run-time.

```
constexpr int foo(int n) {
    if constexpr {
        return 1; // only at compile time
    } else {
        return n * n;
    }
}
```

**Use-case:** Ensures certain operations only happen during constant evaluation (e.g. static initializations). (Not widely used yet, but adds precision to `constexpr` logic.)

- 5 **Deducing `this` (explicit object parameters)** – **Purpose:** Allows writing member functions that deduce the explicit object parameter, e.g. `this` `auto& self`, enabling generic code inside classes.

```
struct S {  
    void print(this auto& self) {  
        std::cout << typeid(self).name() << "\n";  
    }  
};
```

# Summary Table

| Standard     | Feature                        | Why we like it                                                       |
|--------------|--------------------------------|----------------------------------------------------------------------|
| <b>C++11</b> | Move semantics                 | Performance (enables <code>std::move</code> , no unnecessary copies) |
|              | <code>std::unique_ptr</code>   | RAII memory management (avoids manual delete)                        |
|              | auto type deduction            | Concise code, type safety (auto-tracks type)                         |
|              | Range-based for                | Easier iteration (cleaner <code>for</code> over containers)          |
|              | Lambdas                        | In-place callbacks (improves clarity, less boilerplate)              |
| <b>C++14</b> | <code>std::make_unique</code>  | Safer pointer creation, exception-safe new                           |
|              | Relaxed <code>constexpr</code> | More compile-time code (loops, branches)                             |
|              | auto return type               | Function return type deduction (brevity)                             |
|              | Generic lambdas                | Polymorphic lambdas (auto parameters)                                |
|              | <code>decltype(auto)</code>    | Exact return-type forwarding (incl. references)                      |
| <b>C++17</b> | <code>std::string_view</code>  | Zero-copy string handling (views, substrings)                        |
|              | <code>if constexpr</code>      | Compile-time branching in templates                                  |
|              | <code>std::optional</code>     | Expresses value or absence                                           |
|              | <code>std::variant</code>      | Type-safe union (flexible data type)                                 |
|              | Structured bindings            | Unpack tuples/structs succinctly                                     |
| <b>C++20</b> | Concepts                       | Constrained templates (readability, early errors)                    |

| Standard     | Feature                         | Why we like it                                     |
|--------------|---------------------------------|----------------------------------------------------|
|              | <code>std::format</code>        | Modern formatting                                  |
|              | Ranges                          | Composable algorithms (views + pipelines)          |
|              | Coroutines                      | Async/generator language support                   |
|              | Modules                         | Faster builds & encapsulation (modernize includes) |
|              |                                 |                                                    |
| <b>C++23</b> | <code>std::expected</code>      | Explicit error-return (monadic error handling)     |
|              | <code>std::print/println</code> | Convenient formatted I/O in core (like fmt)        |
|              | <code>std::mdspan</code>        | Native multi-D array views (n-dim indexing)        |
|              | <code>if constexpr</code>       | Compile-time-only branch (refines constexpr)       |
|              | Deducing <code>this</code>      | Generic member functions (auto <code>this</code> ) |